Does the best GTA editing strength μ depend on local support and source quality?

Reproducing GTA (NeurIPS 2024) on `halfcheetah-medium-v2` and testing hypothesis H2

[Author names]

The hypothesis in one slide

GTA edits offline-RL data with one global editing strength μ for every trajectory window.

H2. The best μ for a window depends on two properties of that window:

- **H2a (support):** the better the window's region is covered by the data (*local support* S), the **higher** μ can be.
- **H2b (quality):** the higher the window's own reward (*quality* Q), the **lower** μ should be.
- **H2c (downstream):** choosing μ per window from its support improves TD3+BC compared with a single μ.

Answer so far. H2a **supported** for *how far a window can be edited safely* (effect $+0.18$, 95% CI $[0.14, 0.23]$), weakly for the reward-optimal μ ($+0.06$). H2b **not supported** (quality changes how much reward an edit gains, not how strong it should be). H2c is running (Experiment A).

Outline

1. Setup: data, trained diffusion model, how GTA edits a window
2. Hypothesis H2 made measurable: quality Q, support S_1, S_2, S_3, outcomes, statistics
3. Pilot study (184 windows) and what it changed
4. Main study (400 windows, 5,600 analysed edits)
5. Experiment A: does support-adaptive μ help TD3+BC? (running)
6. Limitations, reproducibility, glossary of every output column

GTA in one slide (Lee et al., NeurIPS 2024)

1. **Stage A, train:** a conditional diffusion model learns short trajectory windows τ given their return $y(\tau)$.
2. **Stage B, edit (augment):** take a real window τ^0, add noise *once* at level $\sigma(\mu K)$ ("partial noising"), then denoise for μK steps while conditioning on the *amplified* return $\hat{y} = \alpha\, y(\tau^0)$ (paper Eq. 8–10).
3. **Stage C, train RL:** add the edited windows (as transitions) to the original data and train any offline-RL algorithm (TD3+BC here).

$\mu \in (0, 1]$	noising ratio = how much of the window is rewritten (exploration). $\mu = 1$: pure noise, nothing of the source survives.
$\alpha > 1$	return multiplier = how hard the edit is pushed to higher reward (exploitation). We keep $\alpha = 1.3$ throughout.
K	number of diffusion noise levels, $K = 128$.

This work studies Stage B: whether μ should be chosen per window.

Data used: `halfcheetah-medium-v2` (D4RL)

Source	D4RL `halfcheetah-medium-v2`, collected by one "medium" SAC policy
Size	1,000 episodes $\times$ 1,000 steps = **1,000,000 transitions**
State s (17)	8 joint/body positions (qpos without the x-position) + 9 velocities (`qvel`); `s[8]` = forward velocity
Action a (6)	joint torques in $[-1, 1]$
Reward r	forward velocity $- 0.1\|a\|^2$ (MuJoCo HalfCheetah)
Terminals	none (HalfCheetah never falls; episodes end at the 1,000-step time limit)
Other D4RL datasets	not used in this work

The whole dataset was used to (i) train the diffusion model, (ii) build the support index (S_1), and (iii) as the original half of every TD3+BC training set.

The trained diffusion model (Stage A, reproduced)

Network	MixerUnet (Temporal U-Net with MLP-Mixer blocks), 29,628,949 parameters
Diffusion	EDM (Karras et al. 2022): $\sigma_{\text{min}} = 0.002$, $\sigma_{\text{max}} = 80$, $\rho = 7$, $\sigma_{\text{data}} = 1$
Input	one window, 32×24 array, normalised per column (mean/std over the dataset)
Condition	$c = 0.01 \cdot \text{RTG}$, $\text{RTG} = \sum_{k \geq t} 0.99^{k-t} r_k$ to the episode end; dropped with $p = 0.25$ (classifier-free training)
Training	batch 256, Adam lr $3 \cdot 10^{-4}$, cosine warm restarts, **1M steps**; checkpoints every 200k
Used for editing	EMA (exponential moving average) weights of the 1M-step checkpoint
Sampler	stochastic Heun, 128 steps, $S_{\text{churn}} = 80$, $S_{t\,\text{min}} = 0.05$, $S_{t\,\text{max}} = 50$, $S_{\text{noise}} = 1.003$

What GTA edits: a *window*, not an episode

A window starting at step t of an episode is the 32×24 array

$$\tau = \begin{bmatrix} s_t & a_t & r_t \\ \vdots & \vdots & \vdots \\ s_{t+30} & a_{t+30} & r_{t+30} \\ s'_{t+30} & 0 & 0 \end{bmatrix}$$

rows 0–30: 31 real transitions
($17 + 6 + 1 = 24$ columns); row 31: the final next-state (paper App. A.1, "$H + 1$").

- Windows slide with stride 1: one episode has $1000 - 31 + 1 = 970$ windows; the dataset has **970,000** (overlapping).
- Whole 1,000-step episodes are never generated (paper App. A.1: infeasible).
- An edited window is split back into 31 transitions (s_t, a_t, r_t, s_{t+1}) and added to the RL dataset. Windows are **not** stitched back into episodes; TD3+BC only needs transitions.

How one edit is made (exactly as in the code)

1. Normalise: $\tilde{x} = (\tau - m)/sd$ (per column; the model's own normaliser).
2. Condition: $c = 0.01 \cdot \alpha \cdot \text{RTG}$ with $\alpha = 1.3$ (negative RTG would use $2 - \alpha$; never happens here).
3. Start index $k_0 = \lfloor 128(1-\mu) \rfloor$; noise level $\sigma_k = \left(\sigma_{\max}^{1/\rho} + \frac{k}{127}(\sigma_{\min}^{1/\rho} - \sigma_{\max}^{1/\rho})\right)^{\rho}$.
4. **Noise once:** $x \leftarrow \tilde{x} + \sigma_{k_0}\,\varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$ (pure noise if $\mu = 1$).
5. **Denoise** for $k = k_0, \ldots, 127$ (that is, $128 - k_0$ steps): add churn noise ($\gamma = \min(80/128, \sqrt{2} - 1) = 0.414$ while $\sigma \in [0.05, 50]$), one Euler step with $d = (x - D_\theta(x; \sigma, c))/\sigma$, then a Heun correction.
6. Un-normalise, split into 31 transitions.

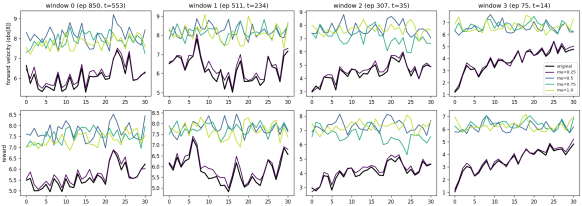
μ	0.1	0.2	0.25	0.3	0.35	0.4	0.45	0.5	0.75
denoising steps $128 - k_0$	13	26	32	39	45	52	58	64	96
noise σ_{k_0} added	0.015	0.081	0.16	0.32	0.54	0.98	1.57	2.43	17.3

μ is **not linear in noise** (data std ≈ 1): $0.25 \to 0.5$ multiplies σ by ≈ 15. Hence a dense μ grid between 0.25 and 0.5.

Implementation choices to be aware of

Guidance strength	config `temperature: 1.0` $\Rightarrow$ score = uncond $+1\cdot$(cond $-$ uncond) = pure conditional, i.e. $w = 0$. The paper reports $w = 2$ (Table 7). Kept at the reproduction's setting for all experiments.
Condition dropout	the upstream network multiplies the condition by a Bernoulli(0.75) mask *also while sampling* (each network call). Kept (upstream behaviour).
Reweighted sampling	off (paper Table 9: not used for halfcheetah-medium).
α	1.3 everywhere (paper's choice for halfcheetah-medium; Table 29 best region).
Hardware	RTX 5090 (PyTorch 2.7.1, CUDA 12.8); simulator checks on CPU with MuJoCo 2.1.

Sanity check: GTA edits on 4 random windows



Top: forward velocity $\mathbf{s}[8]$; bottom: reward, over the 31 steps. Black = original.

win	μ	R_{sim}	dyn MSE	shift
0	0	173.5	–	0
0	0.25	177.6	0.0011	0.13
0	0.5	239.1	0.0063	0.84
0	1.0	227.8	0.0064	1.31
3	0	109.4	–	0
3	0.25	112.9	0.0114	0.14
3	0.5	191.0	0.0021	1.04
3	1.0	194.1	0.0019	1.49

- $\mu = 0.25$: almost no change (+3% reward).
- $\mu \geq 0.5$: +35–75% reward; model reward matches the simulator within 0.5.
- Columns defined on the next slides.

Unit of analysis and quality Q

Unit: one source window i (31 transitions + final state), identified by (episode, start).

Eligible windows (H2 study): starts $0, \ldots, 800$ of every episode $\Rightarrow 801 \times 1000 =$ **801,000**. Later starts are excluded because their RTG (the conditioning value) is cut short by the episode end.

Quality = discounted reward inside the window:

$$q_i = \sum_{t=0}^{30} 0.99^{\,t}\, r_{\text{start}_i + t}, \qquad Q_i = \text{percentile rank of } q_i \text{ in the pool} \in (0, 1].$$

- Inside-the-window reward is used instead of RTG because RTG falls for late windows regardless of behaviour.
- RTG is still what the model is conditioned on (step 2 of the edit).

Local support S_1: nearest neighbours in the data (model-free)

1. Standardise every $(s, a) \in \mathbb{R}^{23}$: $z = (x - \bar{x})/\mathrm{sd}(x)$ over all 1M transitions.
2. For each step t of window i (global transition j_t): distance to the **20th nearest** transition from a **different episode**,

$$d_{20}(j_t) = \text{20th smallest of } \{\|z_{j_t} - z_{j'}\|_2 : \ \text{episode}(j') \neq \text{episode}(j_t)\}.$$

   Same-episode neighbours are excluded: adjacent steps of one run are always near-identical.
3. $\text{s1_dist}_i = \text{median}_t \ d_{20}(j_t)$; $\quad S_{1,i} = \text{percentile rank of } -\text{s1_dist}_i$.

Why it measures density: the k-NN estimate $\hat{p}(x) = k/(N\, V_d\, d_k(x)^d)$ ($d = 23$): smaller $d_{20} \Leftrightarrow$ denser region. It is the inverse of GTA's own *novelty* metric (paper Eq. 12).

Exact brute force (FAISS on CPU, or PyTorch on GPU; identical to 10^{-5}).

Local support S_2: how well the trained model reconstructs the window

1. Normalise the window: $\tilde{x}$. For $\sigma \in \{0.16, 0.5, 2.4\}$ and 4 draws $\varepsilon_n \sim \mathcal{N}(0, I)$, one *unconditional* denoiser call

$$\hat{x} = D_\theta(\tilde{x} + \sigma\varepsilon_n; \sigma, \varnothing), \qquad D_\theta(x;\sigma) = c_{\text{skip}}x + c_{\text{out}}F_\theta(c_{\text{in}}x; c_{\text{noise}})$$

with $c_{\text{skip}} = \frac{1}{\sigma^2+1}$, $c_{\text{out}} = \frac{\sigma}{\sqrt{\sigma^2+1}}$, $c_{\text{in}} = \frac{1}{\sqrt{\sigma^2+1}}$, $c_{\text{noise}} = \frac{1}{4}\ln\sigma$.

2. Error $e_i(\sigma) = \frac{1}{4}\sum_n \frac{\sum_{t,d} M_{td}(\hat{x}-\tilde{x})^2_{td}}{\sum M}$ (mask M drops row 31's action/reward zeros, as in training).
3. $\text{s2_score}_i = -\,\text{mean}_\sigma\ z\text{-score}_{\text{pool}}\big(\ln e_i(\sigma)\big)$; $\quad S_{2,i} =$ percentile rank.

Why: $-\log p(x) \le \int w(\sigma)\,\mathbb{E}_\varepsilon\|D_\theta(x+\sigma\varepsilon;\sigma) - x\|^2 d\sigma + C$ (Kingma et al. 2021; Song et al. 2021). S_2 is a 3-point version. The three σ are the noise levels an edit applies at $\mu \approx 0.25, 0.35, 0.5$. S_2 **is the primary support metric.**

Local support S_3 (validation only) and agreement of the three

S_3: **exact log-likelihood** via the probability-flow ODE $\frac{dx}{d\sigma} = f(x,\sigma) = \frac{x - D_\theta(x;\sigma)}{\sigma}$:

$$\log p_0(x) = \log \mathcal{N}(x_{\sigma_{\max}}; 0, \sigma_{\max}^2 I) + \int_{\sigma_{\min}}^{\sigma_{\max}} \nabla \cdot f(x_\sigma, \sigma)\, d\sigma,$$

divergence by Hutchinson ($v^\top \partial_x f\, v$, one Rademacher probe), dopri5, tol 10^{-4}. Slow: computed on **100** pool windows.

pair (pool, $n = 20{,}000$)	Spearman ρ
S_1 vs S_2	0.79
S_2 vs S_3 ($n = 100$)	0.74
S_1 vs S_3 ($n = 100$)	0.83
Q vs S_1	0.49
Q vs S_2	0.42

- Data-based and model-based support agree; both track the exact likelihood.
- Support and quality are related but not the same (needed for a 2-D design).
- Support spread: $(p_{90} - p_{10})$/median of s1_dist $= 0.52$.

What is measured for every edit

Each edited window $\hat{\tau}$ is replayed in MuJoCo: for each of the 31 rows, set the simulator to $\hat{s}_t$ (x-position $:= 0$, unobserved and irrelevant to dynamics), apply $\hat{a}_t$, read the true next state $f^*(\hat{s}_t, \hat{a}_t)$ and true reward.

`gen_r`	$\sum_t \hat{r}_t$: sum of the 31 rewards *written by the model*
`oracle_r`	$\sum_t r_{\text{env}}(\hat{s}_t, \hat{a}_t)$: the *true* reward of the edited states/actions (paper App. D.3)
`src_oracle_r`	the same for the unedited source window
`gain_rel`	$(\texttt{oracle_r} - \texttt{src_oracle_r}) / \max(\lvert\texttt{src_oracle_r}\rvert, 1)$: relative reward gain
`dyn_mse`	$\frac{1}{31 \cdot 17} \sum_{t,d} \left((f^*(\hat{s}_t, \hat{a}_t) - \hat{s}_{t+1})_d / \mathrm{sd}_d \right)^2$: physics error, dataset-std units (paper Eq. 11)
`shift`	$\sqrt{\frac{1}{31 \cdot 17} \sum_{t,d} ((\hat{s}_t - s_t)_d / \mathrm{sd}_d)^2}$: how far the edit moved from the source

Every (window, μ) is edited **twice** (two noise draws); metrics are averaged over the draws.

From edits to "the best μ" of a window

For window i with metrics $\text{gain}_i(\mu)$, $\text{dyn}_i(\mu)$ on the grid $\mathcal{M}$ and threshold θ:

- $\mu_i^* = \arg\max\{\text{gain}_i(\mu) : \ \mu \in \mathcal{M},\ \text{dyn}_i(\mu) \le \theta\}$ best reward among physically valid edits
- $\mu_{90,i} = \min\{\mu : \ \text{gain}_i(\mu) \ge 0.9 \ \max_{\mu'} \text{gain}_i(\mu')\}$ how much editing the window needs
- μ_i^{safe} = largest μ before dyn_i first exceeds θ (0 if already at $\mu = 0.1$) how far it can be pushed safely

Statistics (per outcome $y \in \{\mu^*, \mu_{90}, \mu^{\text{safe}}\}$, $n = 400$):

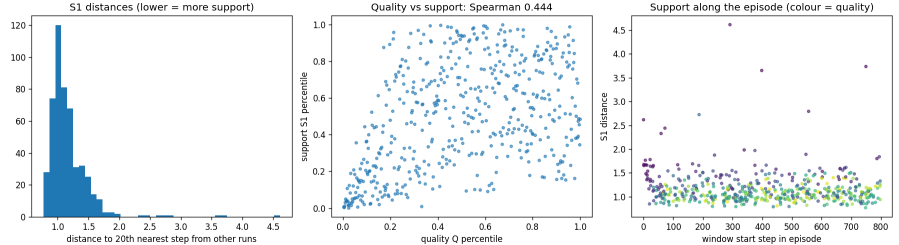
$$y_i = \beta_0 + \beta_Q(Q_i - \tfrac{1}{2}) + \beta_S(S_i - \tfrac{1}{2}) + \beta_{QS}(Q_i - \tfrac{1}{2})(S_i - \tfrac{1}{2}) + e_i$$

β_S = change of y from the least to the best supported window (quality fixed). 95% CI: 2,000 bootstrap resamples of windows. p: 2,000 permutations of S (or Q).

Decision rule (fixed before the main run): **supported** if $\text{CI}(\beta_S) > 0$, $\text{CI}(\beta_Q) < 0$ *and* $\bar{y}(\text{low } Q, \text{high } S) - \bar{y}(\text{high } Q, \text{low } S) \ge 0.1$; partly if only one effect.

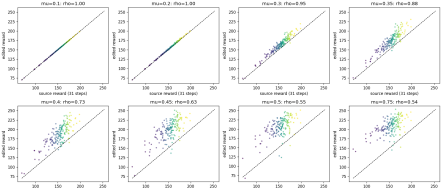
Pilot study: design

Pool	500 random eligible windows (seed 1), support S_1 only (kNN)
Sample	5×5 grid on (Q quintile $\times$ S_1 quintile), up to 8 per cell $\Rightarrow$ **184 windows** (the low-Q/high-S corner was under-filled)
μ grid	0.1, 0.2, 0.3, 0.35, 0.4, 0.45, 0.5, 0.75; 2 draws each $\Rightarrow$ 2,944 edits
Runtime	36 s on the GPU
Purpose	check the pipeline and the outcome definitions before the main run



500 windows. Left: S_1 distances (median 1.09, p_{10}/p_{90} 0.90/1.49). Middle: low-quality windows almost always have low support (Spearman 0.44). Right: least-supported windows are episode starts (the cheetah accelerating from rest).

Pilot results: the first definition was dominated by a ceiling effect



Edited vs source reward per μ (184 windows). At $\mu \geq 0.45$ edits converge to $\approx$ 175–250 whatever the source: rank correlation falls from 1.00 to 0.54.

μ^* analysed with	β_Q [95% CI]	β_S [95% CI]
all μ, $\theta = 0.017$	-0.17 $[-0.24, -0.10]$	$+0.02$ $[-0.05, 0.09]$
all μ, $\theta = 0.0085$	-0.15 $[-0.22, -0.08]$	-0.01 $[-0.08, 0.07]$
$\mu \leq 0.5$, $\theta = 0.017$	-0.02 $[-0.05, 0.00]$	$+0.01$ $[-0.02, 0.05]$
$\mu \leq 0.5$, $\theta = 0.0085$	-0.01 $[-0.05, 0.02]$	$+\mathbf{0.09}$ $[0.04, 0.14]$

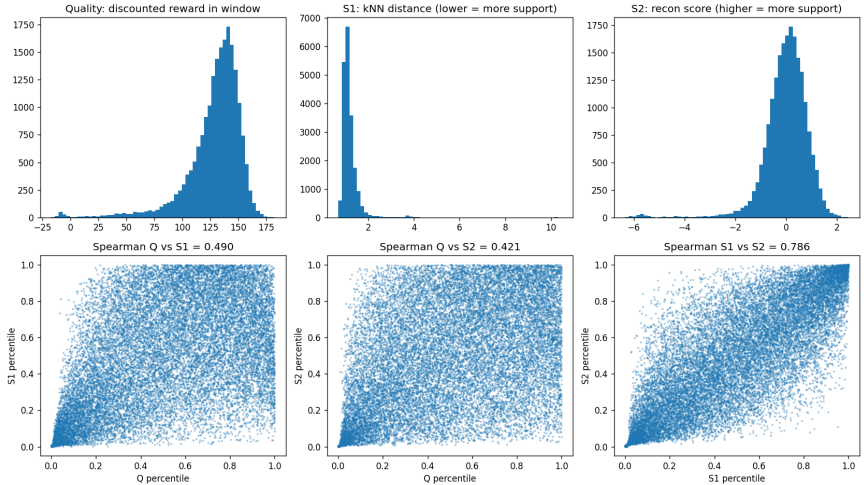
- The apparent quality effect came from $\mu = 0.75$ edits that rewrite every window to a similar high reward.
- Restricting to $\mu \leq 0.5$ (edits that keep part of the source) removes it and reveals a support effect.
- **Fixed for the main run:** $\mu \leq 0.5$, $\theta = 0.0085$ (paper's mean dynamic MSE, Table 19), support S_2.

Main study: which part of the data is used (funnel)

stage	count	how
transitions in the dataset	1,000,000	1,000 episodes $\times$ 1,000
windows (all starts)	970,000	970 per episode
eligible windows (start ≤ 800)	801,000	H2 study only
pool	20,000	uniform random, seed 0; Q, S_1, S_2 computed
of which with S_3	100	random subset
selected windows	**400**	5×5 grid on $Q \times S_2$ quintiles, 16 per cell
edits generated	6,400	400×8 $\mu \times 2$ draws
edits analysed	**5,600**	400×7 μ (≤ 0.5) $\times 2$ draws
simulated transitions	198,400	6,400 $\times$ 31 MuJoCo steps

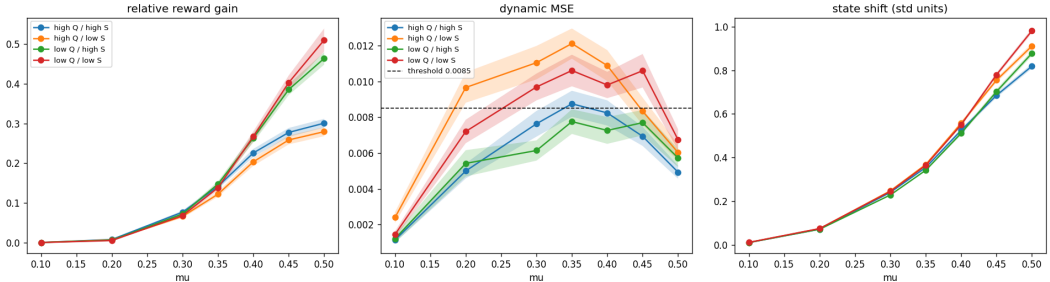
Settings: $\alpha = 1.3$, temperature 1, threshold $\theta = 0.0085$, $\mu \in \{0.1, 0.2, 0.3, 0.35, 0.4, 0.45, 0.5\}$.

The pool: quality and support distributions



Top: histograms of q, s1_dist, s2_score. Bottom: pairwise percentile scatter with Spearman ρ (20,000 windows). Low-quality windows are rarely well supported, which is why sampling is stratified on both axes.

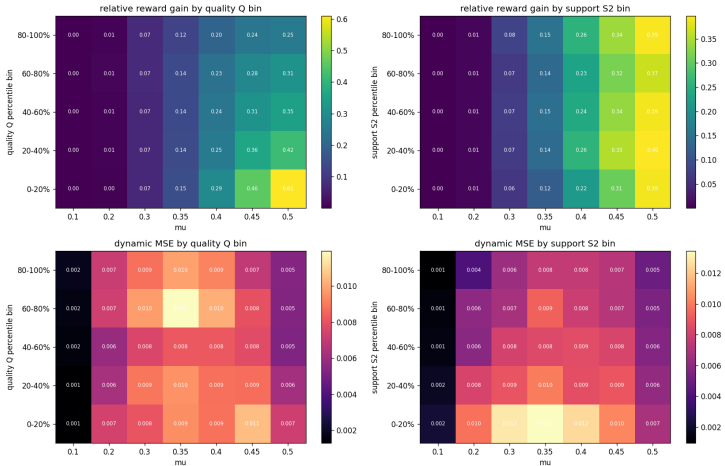
Main result 1: what changes with μ, by quadrant



Quadrants split Q and S_2 at their medians ($\approx$100 windows each); bands $= \pm 1$ s.e.

- **Reward gain** (left) rises with μ for everyone; low-Q windows gain most.
- **Physics error** (middle) is higher for *low-support* windows (orange, red) at every μ; it peaks at $\mu \approx 0.35$ and falls at 0.5.
- **Shift** (right) is the same for all quadrants: equal edit size, unequal damage.

Main result 2: gain follows quality, physics error follows support



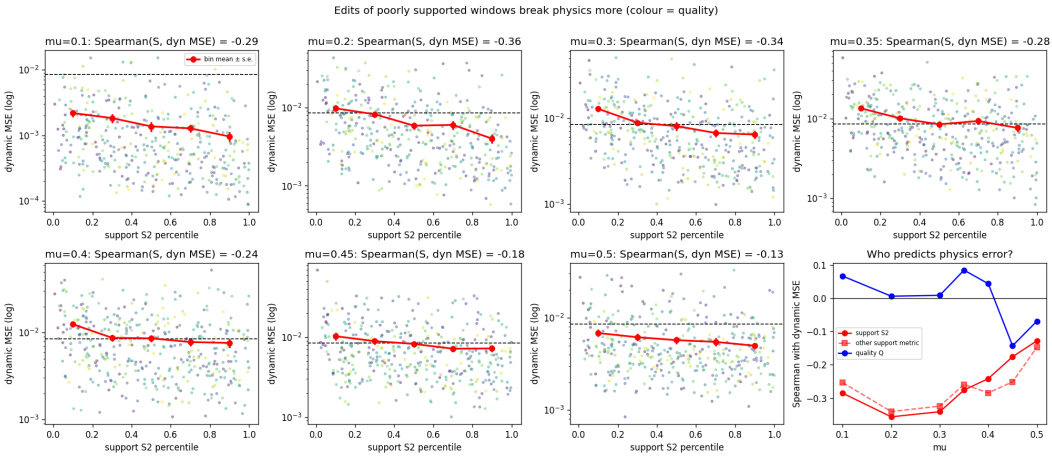
Rows: percentile bins (80 windows each); columns: μ. Top-left: lowest-quality bin gains +61% at $\mu = 0.5$, highest +25%. Bottom-right: lowest-support bin reaches dynamic MSE 0.012–0.013 at $\mu = 0.3$–0.4, highest-support bin 0.006–0.008 ($\approx 2\times$).

Main result 3: correlations at every μ (Spearman, 400 windows)

μ	0.1	0.2	0.3	0.35	0.4	0.45	0.5
S_2 vs physics error	-0.29	-0.36	-0.34	-0.28	-0.24	-0.18	-0.13
S_1 vs physics error	-0.25	-0.34	-0.32	-0.26	-0.28	-0.25	-0.15
Q vs physics error	$+0.07$	$+0.01$	$+0.01$	$+0.09$	$+0.04$	-0.14	-0.07
S_2 vs reward gain	$+0.11$	$+0.11$	$+0.12$	$+0.14$	$+0.09$	$+0.06$	$+0.06$
Q vs reward gain	$+0.13$	$+0.17$	$+0.01$	-0.11	-0.25	-0.51	-0.65
mean reward gain	0.000	0.007	0.071	0.138	0.240	0.331	0.388
mean physics error	0.0015	0.0068	0.0086	0.0098	0.0090	0.0084	0.0058
share over $\theta = 0.0085$	0.02	0.24	0.36	0.43	0.41	0.33	0.18

- Both support metrics predict physics error at every μ; quality does not (mechanism of H2a).
- Quality predicts reward gain (ceiling effect: edits converge towards similar reward).

Main result 3 (visual): support vs physics error per μ



Each dot = one window (colour = quality); red = mean $\pm$ s.e. per support quintile; dashed = θ; log scale. Last panel: correlation of physics error with S_2 (solid), S_1 (dashed) and Q (blue) across μ.

Main result 4: test of H2 (decision rule fixed in advance)

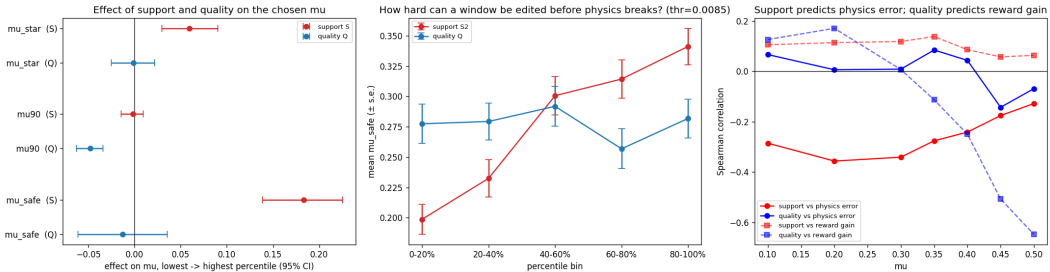
outcome	β_S [95% CI]	p_S	β_Q [95% CI]	gap	verdict
μ^*	$+0.059$ $[0.030, 0.090]$	< 0.001	-0.001 $[-0.025, 0.022]$	$+0.034$	S only, small
μ_{90}	-0.002 $[-0.014, 0.009]$	0.78	-0.048 $[-0.063, -0.034]$	$+0.020$	Q only, small
μ^{safe} †	$+\mathbf{0.183}$ $[0.139, 0.225]$	< 0.001	-0.013 $[-0.061, 0.036]$	$+\mathbf{0.109}$	S only

All three verdicts are "partly supported". gap $= \bar{y}$(low Q/high S) $- \bar{y}$(high Q/low S) (rule needs ≥ 0.1). β_{QS} not significant for any outcome. † μ^{safe} was defined after the first look at this run (exploratory); at $\theta = 0.017$ it gives $\beta_S = +0.110$ $[0.062, 0.155]$, $\beta_Q = -0.030$ $[-0.075, 0.014]$.

Why μ^* moves so little: reward keeps rising up to the top of the grid, so the best valid μ is usually the largest:

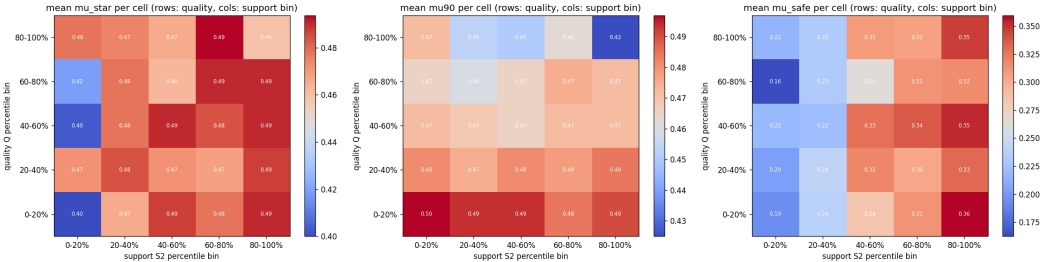
μ^*	0.1	0.2	0.3	0.35	0.4	0.45	0.5
windows	9	2	6	5	13	86	**279**

Main result 5: how far can a window be edited safely?



- Left: effects on each outcome; support (red) is right of zero for μ^* and μ^{safe}, quality (blue) is not below zero except a small μ_{90} effect.
- Middle: mean μ^{safe} by support quintile **0.20, 0.23, 0.30, 0.31, 0.34**; by quality quintile flat at ≈ 0.26–0.29.
- Right: support predicts physics error, quality predicts reward gain.

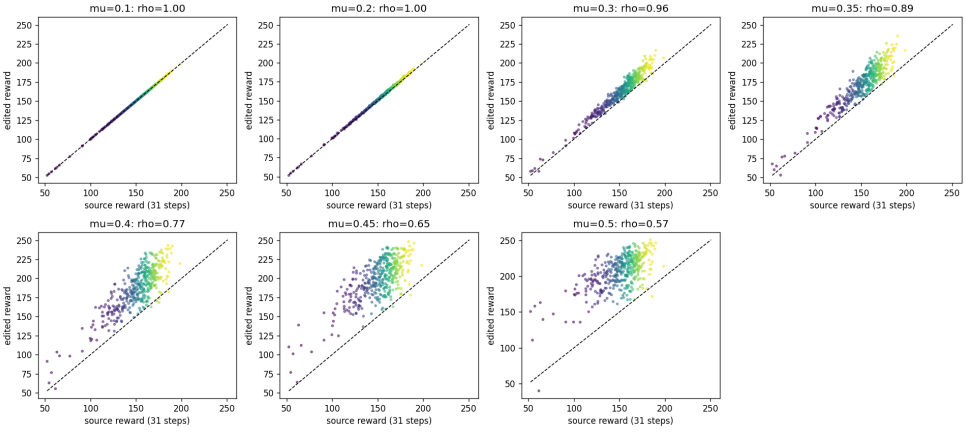
Main result 6: best-μ maps over the 5×5 design



Each cell = mean over its 16 windows. Rows: quality quintile; columns: support quintile.

- μ^* (left) $\approx$ 0.47–0.49 almost everywhere: pinned at the top of the grid.
- μ^{safe} (right) goes from blue ($\approx$ 0.16–0.24) in low-support columns to red ($\approx$ 0.31–0.36) in high-support columns **in every quality row**.

Check: the ceiling effect in the main run



Edited vs source reward (31 steps) per μ; colour = quality; dashed $y = x$. Edits move towards a common reward level as μ grows, so low-quality windows gain more. This is why reward gain alone cannot measure the best μ.

H2 scorecard (main study)

claim	evidence	result
H2a more support $\Rightarrow$ higher μ	μ^{safe}: $+0.18$ $[0.14, 0.23]$; support predicts physics error at every μ, two independent support metrics (ρ -0.13 to -0.36)	**supported** (safety)
	reward-optimal μ^*: $+0.06$ $[0.03, 0.09]$, below the 0.1 size rule	weak
H2b higher quality $\Rightarrow$ lower μ	$\beta_Q \approx 0$ for μ^* and μ^{safe}; only μ_{90}: -0.05. Quality predicts *reward gain* (ρ down to -0.65)	**not supported**
H2c support-adaptive μ helps TD3+BC	Experiment A (next slides)	running

One-line conclusion: support decides how far a window can be edited before its physics breaks; quality decides how much reward the edit gains, not how strong the edit should be.

Side finding: medium μ (0.3–0.45) gives the least consistent windows (half original, half generated); at $\mu = 0.5$ the model regenerates a more self-consistent window but drifts further (shift 0.8–1.0 std).

Experiment A (H2c): design

Question: if μ is chosen per window from its support, does TD3+BC score higher?

Source windows	**161,291** random windows from all 970,000 (any start, as upstream GTA), seed 0; 161,291 × 31 = 5,000,021 generated transitions. **The same windows in every condition.**
Support	S_2 (4 draws, same σ), quintiles within these windows (32,258 each)
Edit	one edit per window, $\alpha = 1.3$, temperature 1
RL data	1M original + 5M generated transitions per condition

condition	μ for support quintile (lowest → highest)				
`none`	original data only				
`uniform_0.5`	0.5	0.5	0.5	0.5	0.5
`uniform_0.3`	0.3	0.3	0.3	0.3	0.3
`adaptive`	0.20	0.25	0.30	0.35	0.40
`inverted`	0.40	0.35	0.30	0.25	0.20

`adaptive` follows the μ^{safe} curve (0.20→0.34). `uniform_0.3` has the same average μ but no targeting; `inverted` has the same μ values in the wrong order. `uniform_0.5` is the paper's setting.

Experiment A: training and decision rule

Algorithm	TD3+BC, CORL implementation (as the paper, App. C.2)
Networks	actor and two critics, 2 hidden layers $\times$ 256, Adam lr $3 \cdot 10^{-4}$
TD3+BC	$\gamma = 0.99$, $\tau = 0.005$, policy noise 0.2, clip 0.5, actor delay 2, $\alpha_{\text{BC}} = 2.5$, states normalised over the training set
Batch, steps	**1,024** (paper App. C.3), **1M** gradient steps
Evaluation	every 10k steps, 10 episodes, D4RL normalised score; **final score** = mean of the last 5 evaluations (960k–1M)
Seeds	0, 1, 2 for each of the 5 conditions (15 runs)
Reference	paper Table 1: TD3+BC 48.4 (no augmentation), 57.8 (GTA); Table 29: $\mu = 0.5, \alpha = 1.3$: 57.9

Decision rule (fixed before results): H2c **supported** if `adaptive` beats **both** `uniform_0.3` and `inverted` (Welch t-test, $p < 0.05$). Comparisons with `uniform_0.5` and `none` are reported but not part of the rule.

Smoke test (3,000 windows, `adaptive`): simulator reward 161.4 vs 148.8 for the sources, mean physics error 0.0088.

Experiment A: status and results

Status: running (generation of the 4 datasets, then 3 rounds of 5 parallel TD3+BC runs; $\approx$ 7–8 h in total).

condition	final score (mean $\pm$ std, 3 seeds)	physics error of its data	mean μ
`none`	[ ]	–	–
`uniform_0.5`	[ ]	[ ]	0.50
`uniform_0.3`	[ ]	[ ]	0.30
`adaptive`	[ ]	[ ]	0.30
`inverted`	[ ]	[ ]	0.30

Filled from `outputs/h2_rl/rl_summary.md` and `figures/rl_results.png` when the runs finish. With 3 seeds, differences below $\approx$ 1 point are unlikely to be significant; seeds 3–4 can be added without rerunning anything.

Limitations and threats to validity

- **One dataset**, one behaviour policy: support varies only moderately (relative spread 0.52). Results may differ on mixed datasets (e.g. medium-replay).
- **Proxy outcomes**: H2a/H2b are tested on data quality (simulator reward, physics error), not on RL performance. H2c (Experiment A) is the RL test.
- **Threshold dependence**: μ^* and μ^{safe} depend on θ. Reported at 0.0085 (paper's mean) and 0.017; direction unchanged.
- **Analysis choices from the pilot**: $\mu \leq 0.5$ and $\theta = 0.0085$ were fixed after the pilot (different windows), then tested in the main run. μ^{safe} was added after the first main analysis: **exploratory**.
- **Grid ceiling**: μ^* is capped at 0.5 (70% of windows sit there).
- **Two noise draws** per edit; per-window metrics are noisy (averaged over 400 windows in all tests).
- **Guidance** at $w = 0$ instead of the paper's $w = 2$; condition dropout active during sampling (upstream).
- Quality and support are correlated ($\rho \approx 0.42$); the stratified design and the regression with both terms separate them.

Reproducibility

Repository branch `dev-karan`; environment and setup in `README.md`.

command	produces
`python scripts/gta_edit_demo.py -ckpt <1M checkpoint>`	4-window demo (`outputs/gta_demo/`)
`python scripts/h2_run.py pool`	pool of 20k windows with Q, S_1, S_2, S_3 (`outputs/h2/pool.csv`)
`python scripts/h2_run.py sweep`	400 windows $\times$ 8 μ $\times$ 2 draws, simulator metrics (`sweep_parts/`)
`python scripts/h2_run.py analyze -max_mu 0.5 -thr 0.0085`	figures, statistics, verdict (`summary.md`)
`bash scripts/h2_rl_all.sh`	Experiment A: datasets, 15 TD3+BC runs, report (`outputs/h2_rl/`)

Seeds: pool 0, pilot pool 1, sampling seed 0, edit seeds fixed per (μ, draw); checkpoint `model-1000000.pt` (EMA weights). Code: `src/h2/` (`common`, `quality`, `support`, `edit_eval`, `analysis`, `render`, `rl_data`, `rl_train`, `rl_report`).

Appendix A: columns of `pool.csv` and `selected.csv`

`episode`, `start`	window = steps `start`…`start`+30 of episode `episode` (0-based)
`q_return`	q_i: discounted ($\gamma = 0.99$) reward inside the window
`rtg`	discounted return-to-go from `start` to the episode end (the model's condition before $\times 0.01 \times \alpha$)
`s1_dist`	median over the 31 steps of the distance to the 20th nearest other-episode transition (lower = more support)
`s2_err_sigma0.16/0.5/2.4`	one-step denoising error at each σ (mean of 4 draws)
`s2_score`	$-$mean of the z-scored log errors (higher = more support)
`s3_loglik`	exact log-likelihood (100 windows; empty otherwise)
`Q`, `S1`, `S2`	percentile ranks in the pool (higher = better quality / more support)
`q_bin`, `s_bin`	quintile (0–4) of `Q` and of the support used for sampling (selected.csv)
`window`	index 0–399 of the selected window (selected.csv)
`src_oracle_r`	simulator reward of the unedited window (selected.csv)

Appendix B: columns of `sweep.csv`, `best_mu.csv`, `metrics.csv`

`mu`, `draw`	editing strength and noise draw (0 or 1) of this edit; in `metrics.csv` `mu`=0 is the source
`gen_r`, `oracle_r`	model-written vs simulator reward, summed over 31 steps
`dyn_mse`, `shift`	physics error and distance from the source (dataset-std units)
`gain_abs`, `gain_rel`	`oracle_r`−`src_oracle_r`, and that divided by max(|`src_oracle_r`|, 1)
`S`	the support column used in the analysis (here `S2`)
`mu_star`, `mu90`, `mu_safe`	per-window outcomes (definitions on the "best μ" slide), after averaging the 2 draws
`quadrant`	`Q` and `S` split at 0.5 ("high Q / low S" etc.)

Experiment A files: `data/sources.npz` (161,291 windows, `s2_score`, `s_bin`); `data/<condition>.npz` (5,000,021 transitions, per-window `mu`, 5,000-window quality check `check_*`); `runs/<condition>_seed<k>/evals.csv` (`step`, `d4rl_score`).

Appendix C: pilot vs main study at a glance

	pilot	main
pool	500 (seed 1)	20,000 (seed 0)
support for sampling	S_1 (kNN)	S_2 (model)
selected windows	184 (8 per cell, sparse corner)	400 (16 per cell, full)
μ grid	8 values incl. 0.75	same; analysis uses $\mu \leq 0.5$
threshold θ	0.017, then 0.0085	0.0085 (fixed)
support $\rightarrow$ physics error, $\mu \leq 0.45$	ρ -0.27 to -0.40	ρ -0.18 to -0.36
β_S for μ^* ($\mu \leq 0.5$, $\theta = 0.0085$)	$+0.087$ $[0.037, 0.141]$	$+0.059$ $[0.030, 0.090]$
β_Q for μ^* (same)	-0.012 $[-0.045, 0.018]$	-0.001 $[-0.025, 0.022]$

The pilot's support effect replicated in the main run with a different support metric and different windows (smaller, still significant).